



DSA5208 Assignment 1

Client-Centric Consistency Experiment

Matriculation number	Name
A0220825M	Koh Si Xing
A0221663L	Ashley Lee Ruoxin
	Kevin Royce Thomson
GitHub Repo: https://github.com/KohSiXing/DSA5208-Consistency-Models	

Acknowledgements	5
Abstract	6
1. Introduction	7
1.1 Project Background and Objective	7
1.2 Research Questions	7
1.3 Client-Centric Consistency Properties	8
1.3.1 Read-Your-Writes	8
1.3.2 Monotonic Reads	8
1.3.3 Monotonic Writes	8
1.3.4 Writes-Follow-Reads	8
2. Building the Replicated MongoDB System	9
2.1 Chosen Database and Replica-Set Architecture	9
2.2 Software Environment	10
2.2.1 Installation procedure	10
2.3 Deployment Method: Different IP Addresses, Same Port	10
2.4 Database Structure and Test Data	11
3. Consistency Choices and Predictions	12
3.1 MongoDB Consistency Controls	12
3.2 Tested Configurations	13
3.3 Hypothesis	14
4. Experimental Design	16
4.1 Common Experimental Procedure	16
4.2 Evaluation Criteria	17
4.3 Causally Consistent Session Comparison	18
5. Results Under Normal Operation and Replication Delay	20
5.1 Scenario Overview	20
5.2 Read-Your-Writes	21
5.2.1 Predictions	21
5.2.2 Results	22
5.2.3 Result Summary	24
5.3 Monotonic-Reads	24
5.3.1 Predictions	25
5.3.2 Results	25
5.3.3 Result Summary	27
5.4 Monotonic-Writes	27
5.4.1 Predictions	28
5.4.2 Results	28
5.4.3 Result Summary	29
5.5 Writes-Follow-Reads	30
5.5.1 Predictions	30

5.5.2 Results	31
5.5.3 Result Summary	31
5.6 Interpretation and Comparison with Predictions	32
6. Results During Database Node Failures	34
6.1 Scenario Overview	34
6.2 One Disconnected Secondary	35
6.2.1 Predictions	36
6.2.2 Results	36
6.2.3 Result Summary	36
6.3 Two Disconnected Secondaries	37
6.3.1 Predictions	38
6.3.2 Results	38
6.3.3 Result Summary	39
6.4 Primary Failure	39
6.4.1 Predictions	41
6.4.2 Results	41
6.4.3 Result Summary	42
6.5 Interpretation and Comparison with Predictions	42
7. Results During Network Partitions	44
7.1 Scenario Overview	44
7.2 Isolated Secondary	45
7.2.1 Predictions	45
7.2.2 Read-Your-Writes Results	46
7.2.2.1 Result Summary	47
7.2.3 Monotonic-Reads Results	47
7.2.3.1 Result Summary	48
7.2.4 Monotonic-Writes Results	48
7.2.4.1 Result Summary	49
7.2.5 Writes-Follow-Reads Results	49
7.2.5.1 Result Summary	50
7.3 Isolated Primary	51
7.3.1 Predictions	51
7.3.2 Results	52
7.3.3 Result Summary	53
7.4 Interpretation and Comparison with Predictions	54
8. Cross-Scenario Discussion	56
8.1 Comparison of the Four Configurations	56
8.2 Effect of Causally Consistent Sessions	57
8.3 Consistency, Availability, and Durability Trade-Offs	58
9. Limitations	59
9.1 Single-Machine Docker Deployment	59

9.2 Timing and Secondary-Selection Variability	59
9.3 Scope of the Tested Failures and Partitions	59
9.4 Observed Behaviour Versus a Guaranteed Property	59
10. Conclusion	61
References:	62

Acknowledgements

Generative AI tools were used to support the preparation of this report and the refactoring of the original code. *ChatGPT Terra 5.6* was used to improve the language and clarity of the report and to generate network diagrams illustrating the experimental scenarios. *Claude Opus 5.5* was used to refactor the code after the original implementation had been developed from a defined experimental methodology, including improving function reusability and the presentation of experimental outputs. The experimental methodology, implementation approach, and interpretation of results were determined by the team.

Abstract

This report investigates client-centric consistency in a three-node MongoDB replica set using a database node with a different loopback IP address while using the same MongoDB port. It supports the normal-operation, replication-delay, node-failure, failover, and rollback experiments. It also simulates a production environment in which each MongoDB node runs on a separate device with its own IP address.

The report evaluates four client-centric consistency properties: read-your-writes, monotonic reads, monotonic writes, and writes-follow-reads. The experiments compare four combinations of write acknowledgement and read behaviour, as well as operations performed with and without causally consistent sessions. Each property is tested under relevant normal-operation, failure, and partition scenarios.

The results show that a write being acknowledged does not mean that every secondary replica has received it. This occurred under both configuration types, although the risk was greater when the write was confirmed by the primary alone. A client reading from a secondary may therefore observe an older value when replication is delayed or communication between nodes is disrupted. Causally consistent sessions prevented completed reads from returning data older than the client's previous operations. However, when a selected replica could not catch up, the read waited and could time out, demonstrating a trade-off between consistency and availability.

The node-failure and failover experiments highlight the difference between confirming a write on the primary alone and confirming it only after it reaches a majority of nodes. A primary-only write can later disappear if that primary fails or becomes isolated before replication occurs. In contrast, a majority-confirmed write is more durable, although the operation may time out when a majority of nodes is unavailable. Some ordering properties are held in the observed runs, but this does not prove that they are guaranteed under every possible failure scenario.

1. Introduction

1.1 Project Background and Objective

Distributed databases improve availability and fault tolerance by replicating data across multiple nodes. However, replication delays, node failures, and communication disruptions can leave replicas with different versions of the data, creating trade-offs among consistency, availability, durability, and write latency.

This report investigates these trade-offs using a three-node MongoDB replica set deployed in Docker on a single machine. Each node is accessible through a different loopback IP address using the default MongoDB port, 27017. The same deployment supports the normal-operation, replication-delay, node-failure, failover, and network-partition experiments. Network partitions are introduced using `iptables` rules that block communication between selected nodes while preserving client access.

1.2 Research Questions

This report addresses the following research questions:

1. How do MongoDB read and write concern configurations affect the four client-centric consistency properties: read-your-writes, monotonic reads, monotonic writes, and writes-follow-reads?
2. Based on the selected configurations, which client-centric consistency properties are expected to hold before the experiments are conducted?
3. Do the experimental results support these predictions under normal operation, replication delay, database-node failure, primary failover, and network partition?
4. How do causally consistent sessions affect the balance between consistent client observations and the availability of read operations?

1.3 Client-Centric Consistency Properties

Client-centric consistency focuses on the sequence of operations observed by an individual client. The following four properties are evaluated in this report.

1.3.1 Read-Your-Writes

Read-your-writes consistency requires that, after a client successfully writes a value, its later reads do not return an earlier value. For example, after updating a student's status to **Graduate**, the same client should not later read **Undergraduate**.

1.3.2 Monotonic Reads

Monotonic reads require that, once a client has read a particular version of data, it does not later observe an older version. For example, after observing a student at **Year 5**, a later read should not return **Year 1**.

1.3.3 Monotonic Writes

Monotonic writes require a client's writes to be applied in the same order in which the client issued them. A replica may receive the writes later because of replication delay, but it should not apply a later write before an earlier one.

1.3.4 Writes-Follow-Reads

Writes-follow-reads consistency requires a write that depends on an earlier read to preserve that causal order. For example, if a client reads that a course is **ACTIVE** and then writes an enrollment note based on that result, the note should remain causally linked to the course status that was read.

2. Building the Replicated MongoDB System

2.1 Chosen Database and Replica-Set Architecture

This project uses MongoDB, a document-oriented NoSQL database that supports replication through replica sets. As illustrated in Figure 1, the system consists of three MongoDB nodes—`mongo1`, `mongo2`, and `mongo3`—configured as the replica set named `dsa5208`.

At any time, one node acts as the primary and accepts all write operations. The remaining nodes act as secondaries, which replicate the primary's operations and may serve read requests when the client is configured to read from secondaries. If the primary becomes unavailable, the available secondaries can elect a new primary, provided that a majority of replica-set members can communicate.

The three-node design allows the experiments to observe replication delay, stale secondary reads, write acknowledgement behaviour, primary failover, rollback, and recovery after a node rejoins the replica set.

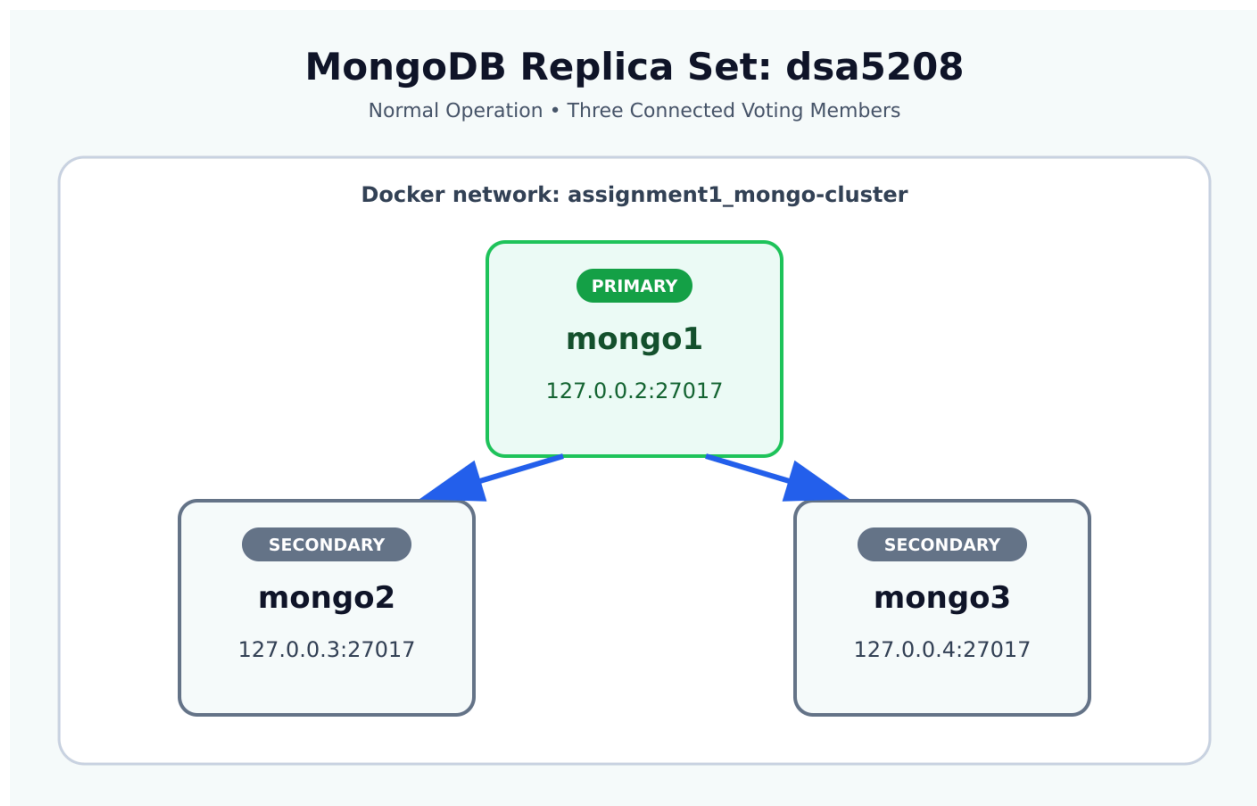


Figure 1: Network Diagram of Replica Set 'dsa5208'

2.2 Software Environment

The experiments were conducted on a local machine using Docker containers. Each MongoDB node ran in a separate Docker container, while Python and PyMongo were used in Jupyter notebooks to execute client operations and record the results. Table 1 below summarises the software environment and versions used for the experiments.

Component	Version
Operating system	Windows 11
Docker	29.7
Python	3.13
MongoDB	8.3
MongoSH	2.10
PyMongo	4.17.0
Pandas	3.0

Table 1: Software Version and Python Packages

The configuration of the replica set and its constituent nodes are stored in a Docker Compose script - docker-compose.yaml. After the containers started, the MongoDB nodes were initialized as a replica set and the client connection was configured with the replica-set name. The notebooks then confirmed that one node was elected primary and that the other two nodes had joined as secondaries before each experiment began.

2.2.1 Installation procedure

Main File: README.md

Docker, Python, MongoDB, and MongoSH were installed using their respective Windows installers. Python, Docker, and MongoSH were added to the system `PATH` environment variable so that their commands could be executed from any directory, including the current project folder. MongoDB Compass was installed alongside MongoDB to provide a graphical interface for inspecting the documents created and updated during the experiments. Finally, the PyMongo driver was installed using `pip`, allowing the Python scripts to connect and interact with the MongoDB replica set.

2.3 Deployment Method: Different IP Addresses, Same Port

Each MongoDB node was assigned a different loopback IP address while using the default MongoDB port, `27017`. The three nodes were accessed as `127.0.0.2:27017`, `127.0.0.3:27017`, and `127.0.0.4:27017`.

This approach was chosen because it more closely resembles a real distributed deployment: each replica is treated as a separate machine with its own network address, while the database service runs on the same standard port. It also allows the client to connect directly to a specific

node for inspection and testing, while the replica-set client automatically routes writes to the current primary.

This deployment method was used for all experiments, including normal operation, replication delay, database-node failure, primary failover, rollback, and network partition. Individual nodes could be paused, restarted, or isolated from their peers without changing the client's ability to identify and access the relevant replica.

2.4 Database Structure and Test Data

The experiments used the **university** database as well as the **student** and **course** collections within the three-node MongoDB replica set. Each student record was stored as a document containing identification details and academic information.

Table 2 describes the fields included when the student document was first created.

Field	Initial value	Purpose
<code>student_id</code>	Supplied through the <code>student_id</code> variable	Identifies the student record.
<code>student_name</code>	Supplied through the <code>name</code> variable	Stores the student's name.
<code>student_status</code>	"Undergraduate"	Provides the starting value for updates used in read-your-writes experiments.
<code>seniority</code>	"Year 1"	Provides the starting value for tracking forward or backward movement in successive reads.
<code>academic_load</code>	"Full Time"	Describes the student's study mode.

Table 2: Fields in Student Document

The following Python code shows how the initial document was constructed:

```
student = {
    "student_id": student_id,
    "student_name": name,
    "student_status": "Undergraduate",
    "seniority": "Year 1",
    "academic_load": "Full Time",
}
```

Figure 2: Sample Data on Student Document

The consistency experiments modified selected fields and compared the values returned by different replica-set members. For example, changing `student_status` from "Undergraduate" to "Graduate" allowed subsequent reads to be checked against the client's write. Advancing `seniority` from "Year 1" to "Year 5" allowed backward reads to be identified.

The report also uses `academic_history`, `course_status`, and `enrollment_note` in the monotonic-write and writes-follow-reads experiments. These fields are not included in the initial document shown above; their starting values and roles are described in the corresponding experimental setups.

Before each experiment, the relevant test fields were reset to their specified baseline values. This provided a consistent starting point for comparing configurations and identifying changes caused by the client operations and controlled disruptions.

3. Consistency Choices and Predictions

3.1 MongoDB Consistency Controls

MongoDB provides several controls that affect when a write is acknowledged and which version of data a read may return. The experiments use write concern, read concern, secondary read preference, and causally consistent sessions to examine their effect on client-centric consistency.

Write concern determines when MongoDB acknowledges a write. With `w="majority"`, the write is acknowledged only after a quorum, i.e. majority of the replica set's voting members, has recorded it. With `w=1`, the write is acknowledged as soon as the current primary stores it, without waiting for replication to secondary nodes.

Read concern determines the level of data that a node may return. `readConcern="majority"` returns data that is majority committed and is therefore unlikely to

be rolled back. However, it does not necessarily return the newest value written by the same client when a read is sent to a lagging secondary. `readConcern="local"` returns the value currently stored by the selected node without waiting for confirmation from other nodes.

The experiments use secondary read preference for selected read operations. This allows reads to be served by either of the secondary nodes and makes replication lag observable. A causally consistent session tracks the order of a client's earlier reads and writes. By default, if a selected secondary cannot provide data that is sufficiently recent for that session, MongoDB waits until the secondary's oplog reaches the session's required timestamp; For the experiment, since we set the `max_time_ms` argument, the operation will return `TIMEOUT` instead of an older value.

3.2 Tested Configurations

Table 3 summarises the four write-concern and read-concern combinations tested in the normal-operation and replication-delay experiments.

Python Configuration	Write Concern	Read Concern	Expected General Behaviour
<code>majority</code>	<code>w="majority"</code>	<code>majority</code>	Stronger write durability and safe reads, but operations may wait when a replica is unavailable or behind.
<code>majority-local</code>	<code>w="majority"</code>	<code>local</code>	Writes are replicated to a majority, while reads return the selected node's locally available value.
<code>w1-majority</code>	<code>w=1</code>	<code>majority</code>	Writes are acknowledged quickly by the primary, while reads return a majority-committed value that may be older than the latest write.
<code>w1-local</code>	<code>w=1</code>	<code>local</code>	Fast write acknowledgement and immediate local reads, but the highest likelihood of observing stale data on a lagging secondary.

Table 3 : MongoDB Configurations used for Experiment

The node-failure and partition experiments focus on the two contrasting configurations, **majority** and **w1-local**, because they clearly demonstrate the trade-off between stronger durability and faster primary-only acknowledgement.

3.3 Hypothesis

The same consistency configuration may behave differently under replication delay, node failure, or network partition. Experiments are therefore developed separately for each scenario, considering the read and write concerns, causal consistency session, and whether the original primary remains active.

The detailed predictions are presented alongside the corresponding experiments, as outlined in Table 4.

Scenario group	Experiments	Focus of predictions
Normal operation and replication delay	Stall replication on one secondary while the remaining nodes continue operating.	Whether reads preserve read-your-writes and monotonic reads, and whether write ordering is maintained.
Node failures	Make one secondary unavailable, make both secondaries unavailable, or terminate the primary.	Whether operations can continue and whether acknowledged writes survive failover.
Network partitions	Isolate a secondary or isolate the primary from the other two members.	Whether reads become stale, writes can be acknowledged, and divergent histories lead to rollback.

Table 4: Experimental Conditions and Focus

For each experiment, predictions are compared with the observed results. Observing a consistency property in one scenario does not establish a guarantee under all conditions. The cross-scenario discussion brings these comparisons together.

4. Experimental Design

The experiments examined how MongoDB's consistency settings affected client observations under replication delay, node failures, and network partitions. Each experiment used a defined starting state and a sequence of reads or writes designed to test a particular consistency property.

4.1 Common Experimental Procedure

Table 5 summarises the procedure followed across the experiments. The timing of client operations and disruptions varied according to the scenario.

Step	Procedure	Purpose
1. Reset baseline data	Reset the relevant document fields and establish the required starting state across the replica set.	Reduce interference from earlier experiments.
2. Select the configuration	Set the write concern, read concern, read preference, and session setting.	Define the configuration being tested.
3. Run the scenario	Perform the specified reads and writes, introducing replication stalls, node failures, or network partitions at the planned point.	Test the property under controlled conditions.
4. Record outcomes	Capture returned values, acknowledgements, timeouts, and available evidence of elections or rollback.	Identify how the client and database responded.
5. Restore the cluster	Resume replication, restart affected nodes, or remove partition rules, then inspect the recovered data.	Check convergence and write survival.
6. Compare with predictions	Assess the recorded outcomes using the criteria in Section 4.2.	Determine whether the observations support the predictions.

Table 5: Experimental Procedures

For delayed-secondary experiments, replication was stalled before updates were issued. For primary-failure experiments, replication to the secondaries was interrupted, a write was attempted, while the primary was then terminated. Partition experiments blocked communication between selected members while preserving client access.

The replication-delay experiments examined all four read/write concern combinations. Node-failure and partition experiments focused on majority/majority and `w:1/local`.

4.2 Evaluation Criteria

Table 6 defines the violation criteria used across **all experimental scenarios**. Each experiment was assessed against the properties relevant to its operation sequence. The examples illustrate possible violations; they do not imply that every violation occurred.

Consistency property	What counts as a violation	Illustrative violation
Read-your-writes	A completed read does not reflect the client's earlier acknowledged write, with no subsequent update explaining its absence.	The client writes "Graduate" but later reads "Undergraduate".
Monotonic reads	A later completed read returns an older version than one the client has already observed.	The client reads "Year 5" and later reads "Year 1".
Monotonic writes	A later write is applied before an earlier write from the same client, or takes effect on a history missing the earlier acknowledged write.	A later PhD entry survives failover while the earlier acknowledged M.Sc. entry is lost.
Writes-follow-reads	A dependent write is applied to a history that does not include the state previously read by the client.	A client reads a course-state update, then writes a dependent note on a history that never incorporated that update.

Table 6: Client-Centric Consistency Properties

These criteria distinguish stale data from ordering violations. Repeatedly reading "Year 1" after an update to "Year 5" is stale, but it does not demonstrate a monotonic-read violation unless the client has already observed a newer version. Likewise, writing a note based on an old course status does not itself demonstrate a writes-follow-reads violation.

Table 7 defines the other outcomes recorded during the experiments.

Outcome	Meaning and evaluation
Stale read	A read returns an older version than a known later state reached during the experiment.
Acknowledged write	The client receives confirmation that the requested write concern has been satisfied. This does not necessarily mean every replica contains the write.
Timeout	An operation does not complete within its configured time limit. A read timeout returns no value and is recorded separately from a consistency violation.
Write durability	A write persists despite a failure. The experiments assess this by checking whether writes survive the tested failover and recovery scenarios. Immediate read-back alone does not establish durability.
Rollback	A write previously present on the old primary is removed when that member reconciles with the surviving replica-set history.

Table 7: Outcomes Seen in this Experiment

A write timeout was treated as an **uncertain outcome**. The write might already have been applied and could subsequently survive or be rolled back. MongoDB does not undo an applied modification merely because its write concern times out.

Finally, observing no violation was interpreted as evidence for the tested sequence, rather than proof of a general guarantee.

4.3 Causally Consistent Session Comparison

The read-your-writes, monotonic-read, and writes-follow-reads experiments compared operations with and without an explicit causally consistent session under replication delay and isolated-secondary partitions.

Table 8 explains the purpose of this comparison.

Setting	How related operations are handled	What the comparison examines
Without an explicit causal session	Operations are not linked through a shared causal session. Successive	Whether the selected read and write concerns alone prevent stale or backward reads.

	secondary reads may observe different replication states.	
With an explicit causal session	Related operations carry session timing information. Later reads must satisfy the applicable timestamp requirement.	Whether the client avoids older observations, and whether this causes waiting or timeouts.

Table 8: Explanation on Casual Consistency Session

In these experiments, a delayed or isolated secondary might be unable to satisfy the session's required timestamp. The comparison therefore considered both **consistency**, through the values returned by completed operations, and **availability**, through whether operations completed during the disruption.

The session comparison did not apply to every experiment. Monotonic-write and primary-failure experiments also examined write ordering and survival independently.

Session usage was interpreted together with the read and write concerns. MongoDB documents all four causal-consistency guarantees with durability when causal sessions use majority read and write concerns. Other combinations may preserve properties in particular scenarios without providing the same complete guarantee.

5. Results Under Normal Operation and Replication Delay

5.1 Scenario Overview

Main File: Assignment1.1-combined_cleaned.ipynb

These experiments examined client-centric consistency while the primary and one secondary continued operating normally and replication on the other secondary was deliberately stalled. This created different data versions across the replica set without introducing a primary failure or network partition.

Table 9 summarises the experimental setup.

Component	Role and behaviour
Replica set	Three-member replica set named <code>dsa5208</code> .
mongo1	Primary; Usually elected as the primary node by the voting members and accepted client writes throughout the experiments.
mongo2	Secondary; continued replicating updates normally.
mongo3	Secondary; application of replicated updates was stalled using the <code>rsSyncApplyStop</code> failpoint.
Secondary reads	Used <code>readPreference: secondary</code> , allowing reads to be served by secondaries with different replication states.
Configurations (Write/Read)	majority/majority, majority/local, <code>w: 1/majority</code> , and <code>w: 1/local</code> .

Recovery	The failpoint was disabled so that mongo3 could re-sync and apply the updates.
-----------------	--

Table 9: Experimental Properties

Here, the cluster remained operational throughout the tests, but mongo3 deliberately lagged behind. The experiments therefore examined the effect of replication delay within an otherwise functioning replica set.

In the following tables, configuration names are written as **write concern/read concern**.

5.2 Read-Your-Writes

This experiment tested whether successive secondary reads returned an older value after the client had observed a newer one.

Initially, all three nodes stored **seniority**: "Year 1". The experiment followed four stages:

- **Stall replication on mongo3:** prevent mongo3 from applying new updates, then update **seniority** sequentially from "Year 1" to "Year 5" through the primary.
- **Without a causal session:** read repeatedly from the secondaries and check whether any value was older than the newest value already observed.
- **With a causal session:** start a new causal session and repeat the secondary reads, checking for backward reads within that session. No additional updates were made.
- **Restore replication:** resume mongo3's replication after completing the reads.

The procedure was repeated for all four write/read concern configurations. Different values distinguish the two writes; each stage checks whether reads reflect its own acknowledged update.

5.2.1 Predictions

Table 10 presents the predictions for this scenario, where the primary remained unchanged.

Write concern	Read concern	Without causal session	With causal session
Majority	Majority	✗ May fail	✓ Expected to hold

Majority	Local	✗ May fail	✓ Expected to hold
w: 1	Majority	✗ May fail	✓ Expected to hold
w: 1	Local	✗ May fail	✓ Expected to hold

Table 10: Read-Your-Writes Predictions

Without a session, a secondary could return data older than the client's update. With a causal session, reads were expected to return the updated value or time out if the selected secondary could not satisfy the session requirement.

5.2.2 Results

Table 11 summarises the recorded results. Each row summarises ten read attempts.

Session setting	Configuration (write/read)	Observed reads	Assessment	Matches prediction?
No causal session	Majority/majority	Alternating reads of "Undergraduate" and "Graduate"	Violation observed	✓ Yes
No causal session	Majority/local	Alternating reads of "Undergraduate" and "Graduate"	Violation observed	✓ Yes
No causal session	w: 1/majority	10 "Undergraduate"	Violation observed	✓ Yes
No causal session	w: 1/local	Alternating reads of "Undergraduate" and "Graduate"	Violation observed	✓ Yes

Causal session	Majority/majority	5 "Post Doc"; 5 timeouts	No violation observed	✓ Yes
Causal session	Majority/local	6 "Post Doc"; 4 timeouts	No violation observed	✓ Yes
Causal session	w: 1/majority	6 "Post Doc"; 4 timeouts	No violation observed	✓ Yes
Causal session	w: 1/local	4 "Post Doc"; 6 timeouts	No violation observed	✓ Yes

Table 11: Read-Your-Writes Results

All four configurations produced stale reads without a causal session. The displayed causal-session sequence returned no stale values, supporting the prediction for this scenario, although four reads could not complete during the replication stall.

Figure 3 illustrates the stale reads summarised in Table 11. The left panel shows reads returning both "Graduate" and the older "Undergraduate" value. The right panel shows w: 1/majority returning "Undergraduate" for all ten reads. Both patterns demonstrate read-your-writes violations because the client read an older value after its update was acknowledged.

Alternating values in all except w1-majority	Stalled values for w1-majority																																												
Wrote student_status: Graduate Reads (no session): <table> <tr> <th>read</th><th>value</th></tr> <tr><td>1</td><td>Undergraduate</td></tr> <tr><td>2</td><td>Graduate</td></tr> <tr><td>3</td><td>Undergraduate</td></tr> <tr><td>4</td><td>Graduate</td></tr> <tr><td>5</td><td>Graduate</td></tr> <tr><td>6</td><td>Undergraduate</td></tr> <tr><td>7</td><td>Undergraduate</td></tr> <tr><td>8</td><td>Undergraduate</td></tr> <tr><td>9</td><td>Undergraduate</td></tr> <tr><td>10</td><td>Graduate</td></tr> </table>	read	value	1	Undergraduate	2	Graduate	3	Undergraduate	4	Graduate	5	Graduate	6	Undergraduate	7	Undergraduate	8	Undergraduate	9	Undergraduate	10	Graduate	Wrote student_status: Graduate Reads (no session): <table> <tr> <th>read</th><th>value</th></tr> <tr><td>1</td><td>Undergraduate</td></tr> <tr><td>2</td><td>Undergraduate</td></tr> <tr><td>3</td><td>Undergraduate</td></tr> <tr><td>4</td><td>Undergraduate</td></tr> <tr><td>5</td><td>Undergraduate</td></tr> <tr><td>6</td><td>Undergraduate</td></tr> <tr><td>7</td><td>Undergraduate</td></tr> <tr><td>8</td><td>Undergraduate</td></tr> <tr><td>9</td><td>Undergraduate</td></tr> <tr><td>10</td><td>Undergraduate</td></tr> </table>	read	value	1	Undergraduate	2	Undergraduate	3	Undergraduate	4	Undergraduate	5	Undergraduate	6	Undergraduate	7	Undergraduate	8	Undergraduate	9	Undergraduate	10	Undergraduate
read	value																																												
1	Undergraduate																																												
2	Graduate																																												
3	Undergraduate																																												
4	Graduate																																												
5	Graduate																																												
6	Undergraduate																																												
7	Undergraduate																																												
8	Undergraduate																																												
9	Undergraduate																																												
10	Graduate																																												
read	value																																												
1	Undergraduate																																												
2	Undergraduate																																												
3	Undergraduate																																												
4	Undergraduate																																												
5	Undergraduate																																												
6	Undergraduate																																												
7	Undergraduate																																												
8	Undergraduate																																												
9	Undergraduate																																												
10	Undergraduate																																												

Figure 3: Read-Your-Writes Results without Causal Consistency Session

```
Wrote student_status:      Post Doc
Reads (causal consistency session):
read    value
1  TIMEOUT
2 Post Doc
3  TIMEOUT
4 Post Doc
5 Post Doc
6 Post Doc
7  TIMEOUT
8 Post Doc
9  TIMEOUT
10 Post Doc
```

Figure 4: Read-Your-Writes Results with Causal Consistency Session

5.2.3 Result Summary

The results matched the predictions in all four configurations. Without a causal session, reads returned **"Undergraduate"** after the client had written **"Graduate"**, demonstrating read-your-writes violations. This occurred because write acknowledgement did not ensure that every secondary had applied the update, so a read could reach an outdated replica.

With causal sessions, all completed reads either returned **"Post Doc"**, or a TIMEOUT. This also matched the prediction: a secondary unable to satisfy the session's required timestamp had to wait rather than return an older value. When it could not catch up within the time limit, the read timed out. This is reflected in the results shown in Figure 4.

5.3 Monotonic-Reads

This experiment tested whether successive secondary reads returned an older value after the client had observed a newer one.

Initially, all three nodes stored **seniority: "Year 1"**. The experiment followed four stages:

- **Stall replication on mongo3:** prevent mongo3 from applying new updates, then update **seniority** sequentially from **"Year 1"** to **"Year 5"** through the primary.
- **Without a causal session:** read repeatedly from the secondaries and check whether any value was older than the newest value already observed.

- **With a causal session:** start a new causal session and repeat the secondary reads, checking for backward reads within that session. No additional updates were made.
- **Restore replication:** resume mongo3's replication after completing the reads.

The procedure was repeated for all four write/read concern configurations.

5.3.1 Predictions

Table 12 presents the predictions for this scenario, where the primary remained unchanged.

Write concern	Read concern	Without causal session	With causal session
Majority	Majority	✗ May fail	✓ Expected to hold
Majority	Local	✗ May fail	✓ Expected to hold
w: 1	Majority	✗ May fail	✓ Expected to hold
w: 1	Local	✗ May fail	✓ Expected to hold

Table 12: Monotonic-Reads Predictions

Without a session, reads could move backwards when served by secondaries at different replication stages. With a causal session, later reads were expected to preserve the state already observed within that session or time out.

5.3.2 Results

Table 13 summarises the saved notebook outputs. Each row represents ten read attempts.

Session	Configuration (write/read)	Read Values	Stale reads	Timeout	Matches prediction?
No causal session	Majority/majority	"Year 1" and "Year 5"	Yes	No	✓ Yes

No causal session	Majority/local	"Year 1" and "Year 5"	Yes	No	✔ Yes
No causal session	w: 1/majority	"Year 1" only	No	No	⚠ Inconclusive*
No causal session	w: 1/local	"Year 1" and "Year 5"	Yes	No	✔ Yes
Causal session	Majority/majority	"Year 5" only	No	Yes	✔ Yes
Causal session	Majority/local	initial "Year 1" reads, followed by "Year 5"	No	Yes	✔ Yes
Causal session	w: 1/majority	"Year 5" only	No	Yes	✔ Yes
Causal session	w: 1/local	"Year 5" only	No	Yes	✔ Yes

Table 13: Monotonic-Reads Results

Regression values in all except w1-majority	Stalled values for w1-majority
---	--------------------------------

Reads (no session):			Reads (no session):		
read	value		read	value	
1	Year	5	1	Year	1
2	Year	1	2	Year	1
3	Year	1	3	Year	1
4	Year	1	4	Year	1
5	Year	1	5	Year	1
6	Year	5	6	Year	1
7	Year	1	7	Year	1
8	Year	5	8	Year	1
9	Year	5	9	Year	1
10	Year	5	10	Year	1

Figure 5: Monotonic Reads Results without Causal Consistency Session

5.3.3 Result Summary

Without a causal session, the results matched the predicted risk of violations for majority/majority, majority/local, and `w: 1/local`. Reads returned "Year 1" after "Year 5" because successive requests could reach secondaries at different replication stages.

The prediction comparison was inconclusive for `w: 1/majority`. All reads returned "Year 1", so no violation occurred, but the client never observed a newer value from which a later read could regress.

With causal sessions, all four configurations matched the prediction: no backward reads occurred. Once the session observed "Year 5", a secondary unable to satisfy that requirement had to wait or time out rather than return "Year 1". The initial "Year 1" reads under majority/local were valid because they occurred before the session observed "Year 5".

5.4 Monotonic-Writes

This experiment tested whether sequential writes retained their order while mongo3's replication was stalled.

Initially, all three nodes stored `academic_history: ["B.Sc. Computer Science"]`. The experiment followed four stages:

- **Stall replication on mongo3:** prevent mongo3 from applying new updates while the other nodes continue operating.

- **Write sequentially:** append "M.Sc. Data Science" and then "PhD Information Systems" through the primary.
- **Read during the stall:** inspect the primary and mongo3 directly, then perform secondary reads to compare the academic histories.
- **After replication resumed:** read mongo3's academic history to check whether it contained the bachelor's, master's, and doctoral entries in the correct order.

The procedure was repeated for all four configurations. No explicit causal session was used.

5.4.1 Predictions

Table 14 presents the predictions. No explicit causal session was used in this experiment.

Write concern	Read concern	Without causal session
Majority	Majority	✓ Expected to hold
Majority	Local	✓ Expected to hold
w: 1	Majority	✓ Expected to hold
w: 1	Local	✓ Expected to hold

Table 14: Monotonic-Writes Predictions

Write order was expected to be preserved because both updates were issued sequentially to the same primary. A delayed secondary could return an older array while still applying the updates in the correct order after replication resumed.

5.4.2 Results

Table 15 summarises the saved notebook outputs. **Baseline** refers to ["B.Sc. Computer Science"]. **Complete** refers to ["B.Sc. Computer Science", "M.Sc. Data Science", "PhD Information Systems"], with the qualifications appearing in the order they were written.

Configuration (write/read)	Primary read during stall	Secondary reads	mongo3 after recovery	Results	Matches prediction?

Majority/majority	Complete	Both complete and baseline reads	Complete	No violation observed	✓ Yes
Majority/local	Complete	Both complete and baseline reads	Complete	No violation observed	✓ Yes
w: 1/majority	Baseline	All baseline reads	Complete	No violation observed	✓ Yes
w: 1/local	Complete	Both complete and baseline reads	Complete	No violation observed	✓ Yes

Table 15: Monotonic-Writes Results

5.4.3 Result Summary

All four configurations matched the prediction: no monotonic-write violation was observed. After mongo3 resumed replication, its academic-history array contained the bachelor's, master's, and doctoral qualifications in the expected order.

This was expected because the client sent the M.Sc. update first and the PhD update second to mongo1, which remained the primary throughout the experiment. Only mongo3's replication was stalled. When it resumed, the recorded results showed the updates in the same order.

Reads returning only "B.Sc. Computer Science" indicated that the later qualifications were not yet visible, rather than written in reverse order. This experiment tested replication delay; it did not test what would happen if mongo1 failed between the two writes.

5.5 Writes-Follow-Reads

This experiment tested a read-dependent write while mongo3's replication was stalled.

Initially, the course stored `course_status`: "ACTIVE", and the student's `enrollment_note` was None. The experiment followed four stages:

- **Stall replication on mongo3:** prevent mongo3 from applying new updates while the other nodes continue operating.
- **Without a causal session:** read the course status from a secondary, write an enrollment note based on that value through the primary, and inspect the notes on the primary and mongo3.
- **With a causal session:** start a new causal session, read the course status, write a new enrollment note based on that read, and read the note back from a secondary within the same session.
- **Restore replication:** resume mongo3's replication after completing the operations.

The procedure was repeated for all four configurations. The course status remained "ACTIVE" throughout; this experiment did not introduce a cancellation.

5.5.1 Predictions

Table 16 presents the predictions for this setup.

Write concern	Read concern	Without causal session	With causal session
Majority	Majority	✓ Expected to hold	✓ Expected to hold
Majority	Local	✓ Expected to hold	✓ Expected to hold
w: 1	Majority	✓ Expected to hold	✓ Expected to hold
w: 1	Local	✓ Expected to hold	✓ Expected to hold

Table 16: Writes-Follow-Reads Predictions

The course status remained "ACTIVE" throughout the experiment. Since this value was established before the disruption, the dependent note was expected to follow a course state already available to the primary.

Table 17 summarises the saved notebook outputs. Every course-status read returned "ACTIVE".

5.5.2 Results

Configuration (write/read)	Primary note without a session	mongo3 note without a session	Causal session read-back	Results	Matches prediction?
Majority/majority	Note containing "ACTIVE"	Note containing "ACTIVE"*	Session note containing "ACTIVE"	No violation observed	✓ Yes—limited evidence
Majority/local	Note containing "ACTIVE"	Note containing "ACTIVE"*	Session note containing "ACTIVE"	No violation observed	✓ Yes—limited evidence
w: 1/majority	Note containing "ACTIVE"	Note containing "ACTIVE"*	Session note containing "ACTIVE"	No violation observed	✓ Yes—limited evidence
w: 1/local	Note containing "ACTIVE"	None	Session note containing "ACTIVE"	No violation observed	✓ Yes—limited evidence

Table 17: Writes-Follow-Reads Results

5.5.3 Result Summary

The recorded results matched the prediction that writes-follow-reads would hold in this setup: no violation was observed in any configuration, and all causal-session read-backs succeeded without timeouts.

This was expected because the course status was already "ACTIVE" before replication was stalled and never changed afterwards. Every enrollment note therefore depended on information already available to the primary accepting the write. The experiment did not test what would happen if that primary lacked the course state previously read by the client.

The conclusion remains limited because mongo3 returned the new note in three configurations despite the intended replication stall. The stall must be verified before drawing stronger conclusions about behaviour under replication delay.

5.6 Interpretation and Comparison with Predictions

The results generally supported the predictions for replication delay, although some observations provided limited evidence. Table 18 summarises the findings across the four client-centric consistency properties.

Consistency property	Without a causal session	With a causal session	Matches prediction?	Additional note
Read-your-writes	All four configurations returned stale values after an acknowledged update.	Completed reads returned "Post Doc"; other attempts timed out.	✓ Yes	Timeouts indicate reads could not be completed, rather than consistency violations.
Monotonic reads	Three configurations showed backward reads. w: 1/majority returned only "Year 1".	No backward reads occurred; some attempts timed out.	✓ Yes, except ⚠ non-session w: 1/majority	That run never observed a newer value, so it could not demonstrate regression.
Monotonic writes	All configurations preserved the qualification order after recovery.	Not tested.	✓ Yes	Bachelor's-only reads indicated delayed visibility, not reversed write ordering.
Writes-follo w-reads	No violation was demonstrated.	All dependent-note read-backs succeeded.	✓ Yes, with limited evidence	"ACTIVE" existed before the disruption and remained unchanged, so no missing dependency was tested.

Table 18: Summary of Predictions and Observations

The read-your-writes and monotonic-reads results supported the prediction that secondary reads could violate these properties without causal sessions, including under majority/majority. With causal sessions, completed reads preserved the required relationship to earlier operations, although some reads timed out during the replication stall.

Monotonic writes also matched the predictions: both updates were issued sequentially to the primary, and mongo3 contained the qualifications in the correct order after recovery. The writes-follow-reads results provided weaker evidence because the required course state already existed before the disruption. Furthermore, mongo3 returned the new note in three configurations despite the intended stall, so the effectiveness of that disruption requires verification.

6. Results During Database Node Failures

6.1 Scenario Overview

Main File: Assignment1.2-failover_cleaned.ipynb

These experiments examined write availability, read visibility, and write survival during node failures and failover in the three-node `dsa5208` replica set.

Only **majority/majority** and **w:1/local** were tested to focus the comparison on two contrasting settings: majority acknowledgement with majority-committed reads, and primary-only acknowledgement with local reads. All four combinations had already been explored under replication delay. No explicit causal sessions were used, keeping the focus on the selected read/write concerns. Table 19 summarises the various failure scenarios we experimented.

Scenario	How the failure was introduced	Focus of the experiment
One unavailable secondary	Pause one secondary's Docker container while the other two members continue operating.	Whether writes and primary reads continue, and whether the paused node catches up after recovery.
Two unavailable secondaries	Pause both secondary containers, leaving only the primary available.	Whether writes can be acknowledged without a majority and what subsequent primary reads return.
Primary failure and failover	Stop replication to both secondaries, attempt a write, and terminate the primary. Restore replication so that a new primary can be elected.	Whether the earlier write survives and whether the old primary rolls back its local write when it rejoins.

Table 19: Database-node Failure Scenarios

The secondary-failure experiments updated `student_status` from "Undergraduate" to "Graduate". The primary-failure experiment began with ["B.Sc. Computer Science"],

attempted to append "M.Sc. Data Science" before failure, and appended "PhD Information Systems" after failover.

The notebook identified the current primary before each run, so node roles could differ between experiments. In the two-secondary scenario, operations were attempted shortly after pausing the nodes, before the isolated primary was expected to step down. Results were recorded during the disruption and after recovery.

6.2 One Disconnected Secondary

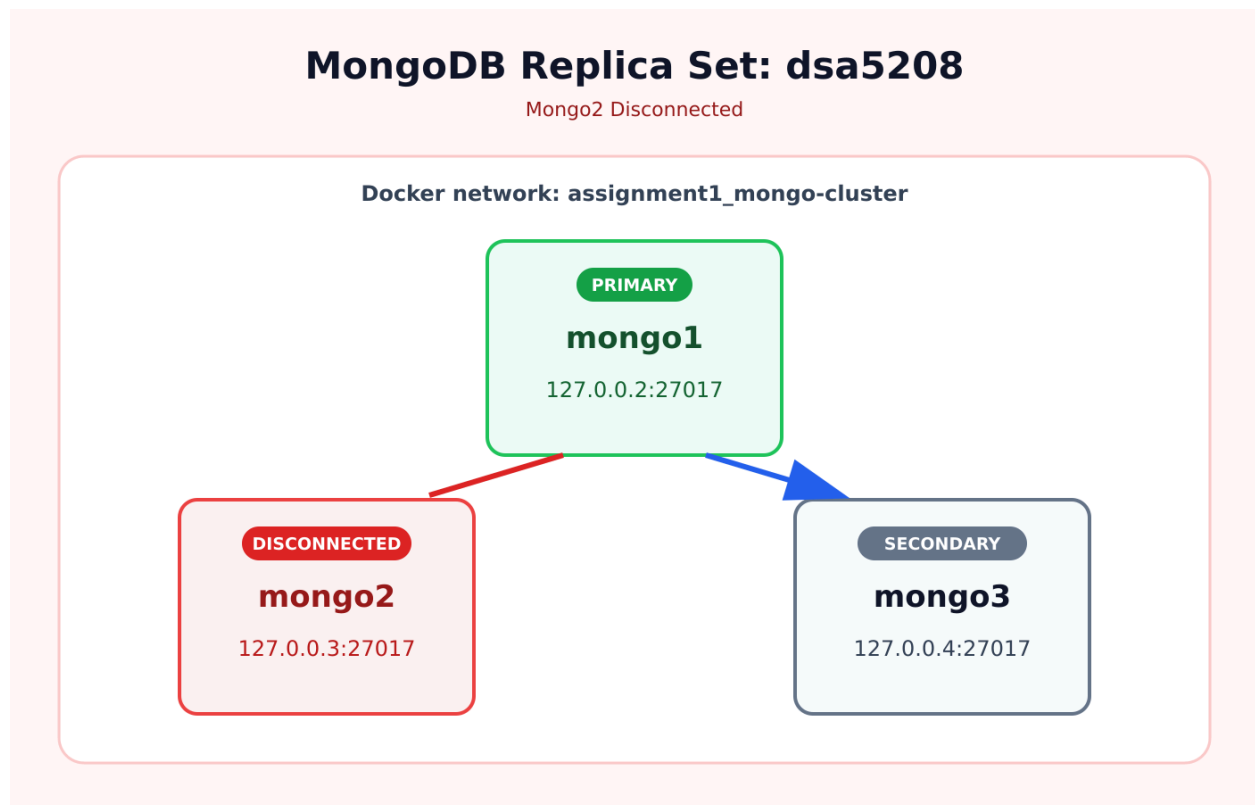


Figure 6: Replica Set with One Disconnected Secondary

Initially, all three nodes stored `student_status: "Undergraduate"`. The experiment followed three stages:

- **Pause one secondary:** pause the mongo2 Docker container while the primary and the other secondary remain running.
- **While the secondary was unavailable:** update `student_status` to "Graduate" through the primary, record the write outcome, and read the value from the primary.
- **After the secondary resumed:** check all three nodes to determine whether they contained "Graduate".

The procedure was repeated for `majority/majority` and `w: 1/local`.

6.2.1 Predictions

Table 20 presents the predictions. With two members still available, the replica set was expected to retain a majority and continue serving writes.

Configuration (write/read)	Expected write outcome	Expected primary read	Expected recovery
Majority/majority	Acknowledged	"Graduate"	All three nodes return "Graduate"
w: 1/local	Acknowledged	"Graduate"	All three nodes return "Graduate"

Table 20: Predictions with One Disconnected Secondary

6.2.2 Results

Table 21 summarises the saved notebook results.

Table 21. Results with one disconnected secondary

Configuration (write/read)	Write outcome	Primary read	Values after reconnection	Matches prediction?
Majority / majority	Acknowledged	"Graduate"	"Graduate" on all three nodes	✓ Yes
w: 1/local	Acknowledged	"Graduate"	"Graduate" on all three nodes	✓ Yes

Table 21: Results with One Disconnected Secondary

6.2.3 Result Summary

Both configurations matched the predictions: the writes were acknowledged, primary reads returned "Graduate", and all three nodes returned "Graduate" after reconnection.

This was expected because, with only one secondary paused, the primary and the remaining secondary still formed a majority. Majority/majority could obtain confirmation from another member, while `w: 1/local` required confirmation only from the primary.

Both configurations completed successfully because the primary remained available. Their identical results do not imply identical protection if the primary fails before a write reaches a secondary.

6.3 Two Disconnected Secondaries

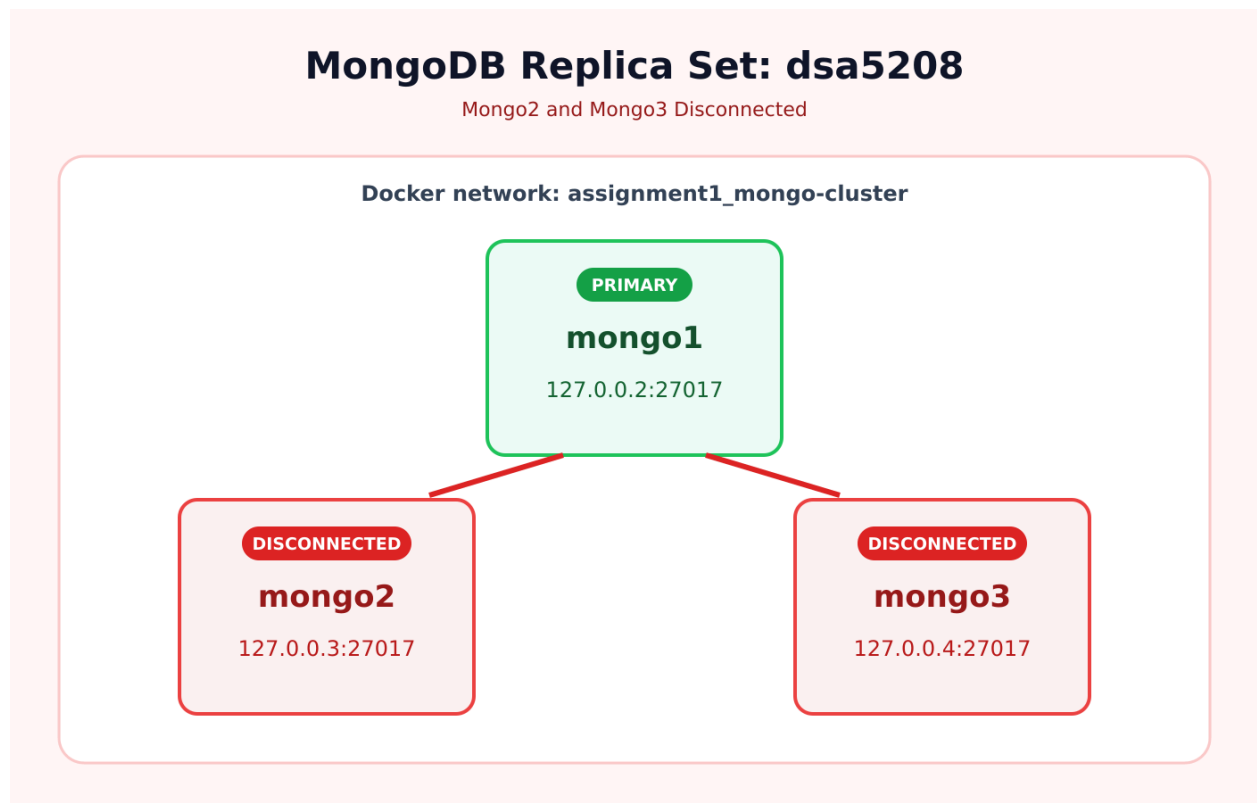


Figure 7: Replica Set with Two Disconnected Secondary

Initially, all three nodes stored `student_status: "Undergraduate"`. The experiment followed three stages:

- **Pause both secondaries:** pause mongo2 and mongo3, leaving mongo1 as the only available member.
- **Before the primary stepped down:** attempt to update `student_status` to `"Graduate"`, record the write outcome, and read the value from the primary.
- **After both secondaries resumed:** check all three nodes to determine whether the update had replicated and they contained `"Graduate"`.

The procedure was repeated for **majority/majority** and `w:1/local`.

6.3.1 Predictions

Operations were attempted shortly after the pause, before the primary was expected to step down. Table 22 presents the predictions for this period.

Configuration (write/read)	Expected write outcome	Expected primary read	Expected recovery
Majority/majority	Write concern times out because no majority is available	<code>"Undergraduate"</code>	The locally applied update replicates after the secondaries resume, provided it survives
<code>w: 1/local</code>	Acknowledged while the primary still accepts writes	<code>"Graduate"</code>	The update replicates after the secondaries resume, provided it survives

Table 22: Predictions with Two Disconnected Secondaries

6.3.2 Results

Table 23 summarises the saved notebook results.

Configuration (write/read)	Write outcome	Primary read	Values after reconnection	Matches prediction?
----------------------------	---------------	--------------	---------------------------	---------------------

Majority/majority	Not confirmed within 3000 ms	"Undergraduate"	"Graduate" on all three nodes	✓ Yes
w:1/local	Acknowledged	"Graduate"	"Graduate" on all three nodes	✓ Yes

Table 23: Results with Two Disconnected Secondaries

6.3.3 Result Summary

Both configurations matched the predictions. Majority/majority timed out and returned "Undergraduate", while w: 1/local acknowledged the update and returned "Graduate". After the secondaries resumed, all three nodes returned "Graduate".

This was expected because the primary could store the update but could not obtain majority confirmation while both secondaries were paused. Majority reads therefore returned the earlier committed value, whereas local reads could return the primary's newly stored "Graduate" value.

The update later replicated successfully, showing that the majority timeout did not cancel it. However, this experiment did not test whether it would survive a primary failure before replication resumed.

6.4 Primary Failure

MongoDB Replica Set: dsa5208

Primary Down • Mongo2 New Primary

Docker network: assignment1_mongo-cluster

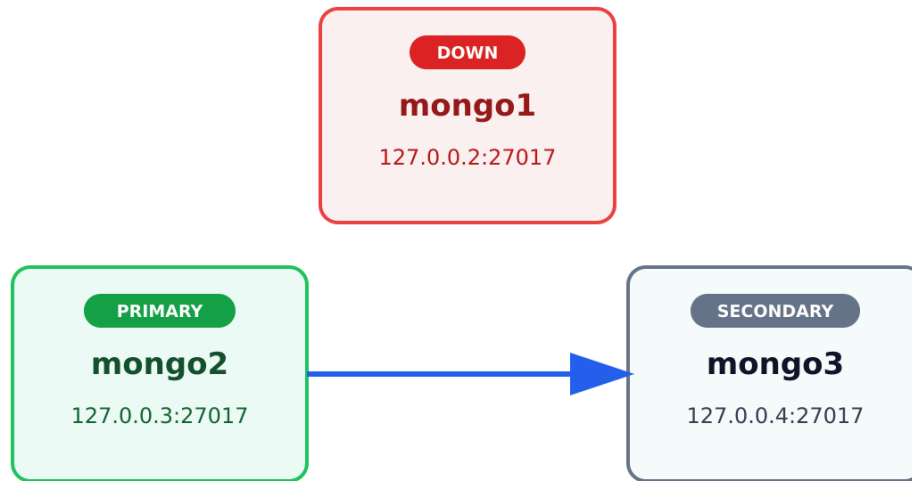


Figure 7: Replica Set with Primary Failure

Initially, all three nodes stored `academic_history: ["B.Sc. Computer Science"]`. The experiment followed four stages:

- **Before primary failure:** stop replication to both secondaries, attempt to append "`M.Sc. Data Science`" through the primary, and record the write outcome and primary read.
- **After terminating the primary:** resume replication on the remaining nodes, wait for a new primary to be elected, and read the document to check whether the M.Sc. entry survived.
- **After failover:** append "`PhD Information Systems`" through the new primary and read the updated history.
- **After restarting the old primary:** inspect its academic history and check its logs for rollback.

The procedure was repeated for `majority/majority` and `w:1/local`.

6.4.1 Predictions

Table 24 presents the predictions.

Configuration (write/read)	Expected M.Sc. write outcome	Expected client read after failover	Expected recovery
Majority/majority	Not confirmed because neither secondary receives the write	["B.Sc. Computer Science"]	The old primary rolls back the unconfirmed M.Sc. entry
w:1/local	Acknowledged by the original primary	["B.Sc. Computer Science"]	The old primary rolls back the acknowledged M.Sc. entry

Table 24: Predictions during Primary Failure

Before termination, direct node checks confirmed that only the original primary contained "M.Sc. Data Science". Both secondaries contained only "B.Sc. Computer Science".

6.4.2 Results

write/read)	M.Sc. write outcome	Client read before failure	Client read after failover , before PhD write	Client read after PhD write	Old primary after rejoining	Rollback detected?	Matches prediction?
Majority/majority	Not confirmed within 3000 ms	["B.Sc. Computer Science"]	["B.Sc. Computer Science"]	["B.Sc. Computer Science", "PhD Information Systems"]	["B.Sc. Computer Science", "PhD Information Systems"]	Yes	 Yes

w: 1/local	Acknowledged	["B.Sc. Computer Science", "M.Sc. Data Science"]	["B.Sc. Computer Science"]	["B.Sc. Computer Science", "PhD Information Systems"]	["B.Sc. Computer Science", "PhD Information Systems"]	Yes	 Yes
------------	--------------	---	----------------------------------	--	--	-----	---

Table 25: Results during Primary Failure

The client wrote "PhD Information Systems" through the new primary after reading the document following failover. After the old primary rejoined, its M.Sc. entry was rolled back, and its history contained the B.Sc. and PhD entries.

Table 25 summarises the saved notebook results.

6.4.3 Result Summary

Both configurations matched the predictions. Under w:1/local, the M.Sc. write was acknowledged and visible before failure but disappeared after failover. Under majority/majority, the write timed out and was not visible through the configured majority reads. Both old primaries rolled back the M.Sc. entry after rejoining.

This occurred because neither secondary received the M.Sc. write before the original primary failed. The newly elected primary therefore contained only [B.Sc.]. With w: 1/local, the client had already received acknowledgement and read [B.Sc., M.Sc.]; with majority/majority, it read only [B.Sc.] even before the failure.

The client subsequently added PhD on the new primary. Under w: 1/local, losing the earlier acknowledged M.Sc. entry while retaining the later PhD entry demonstrated read-your-writes, monotonic-read, and monotonic-write violations. The majority run instead showed that a write can be applied locally and later rolled back without ever receiving the requested acknowledgement.

6.5 Interpretation and Comparison with Predictions

All three failure scenarios matched the predictions. Table 26 summarises how node availability affected acknowledgement, read visibility, and recovery.

Scenario	Majority/majority	w: 1/local	Matches prediction?
One disconnected secondary	Write acknowledged; primary returned "Graduate"; all nodes returned "Graduate" after recovery.	Same observed outcome.	✓ Both
Two disconnected secondaries	Write timed out; majority read returned "Undergraduate"; "Graduate" replicated after reconnection.	Write acknowledged; local read returned "Graduate"; all nodes returned "Graduate" after reconnection.	✓ Both
Primary failure	M.Sc. write timed out and was not visible through majority reads; it was subsequently rolled back.	M.Sc. write was acknowledged and visible before failure, but disappeared after failover and was subsequently rolled back.	✓ Both

Table 26: Predictions-vs-Observations During Node Failures

With one secondary unavailable, the remaining two nodes could still satisfy majority acknowledgement. With both secondaries unavailable, only w: 1 could acknowledge the update while the primary still accepted writes. Majority/majority instead timed out and returned the earlier majority-committed value.

Primary failure exposed the consequence of this difference. Under w: 1/local, the client lost an acknowledged and previously observed M.Sc. entry, while the later PhD entry survived. Under the evaluation criteria in Section 4.2, this demonstrated read-your-writes, monotonic-read, and monotonic-write violations. Under majority/majority, the lost M.Sc. entry had neither received the requested acknowledgement nor appeared in the configured majority reads.

The experiments also showed that **a write timeout leaves the outcome uncertain**: the timed-out update survived when the secondaries resumed, but was rolled back when the primary failed before replication. Matching the predictions therefore means that the observed behaviour was expected; it does not mean that every consistency property was preserved.

7. Results During Network Partitions

7.1 Scenario Overview

Main File: Assignment1.3-partition_cleaned.ipynb

These experiments examined how network partitions affected consistency in the three-node `dsa5208` replica set. Unlike Section 6, where containers were paused or stopped, **all database nodes remained running**. `iptables` firewall rules blocked communication between selected nodes while preserving client access.

Both **majority/majority** and **w: 1/local** were tested, with their purpose listed in the table below.

Scenario	Partition arrangement	Purpose
Isolated secondary	mongo3 was isolated from mongo1 and mongo2. mongo1 remained primary, and mongo2 continued receiving updates.	Test the four client-centric consistency properties when an isolated secondary could still serve stale reads.
Isolated primary	The current primary was isolated from both secondaries. The connected secondaries could elect a new primary.	Examine whether writes accepted by the isolated primary before stepping down were acknowledged and whether they survived reconnection.

Table 27: Network-partition scenarios

The isolated-secondary tests included causal-session comparisons for read-your-writes, monotonic reads, and writes-follow-reads. The monotonic-writes test used no explicit causal session.

After each experiment, the firewall rules were removed to restore communication. The nodes were then checked for convergence for the isolated secondary node and rollback in the isolated-primary tests.

7.2 Isolated Secondary

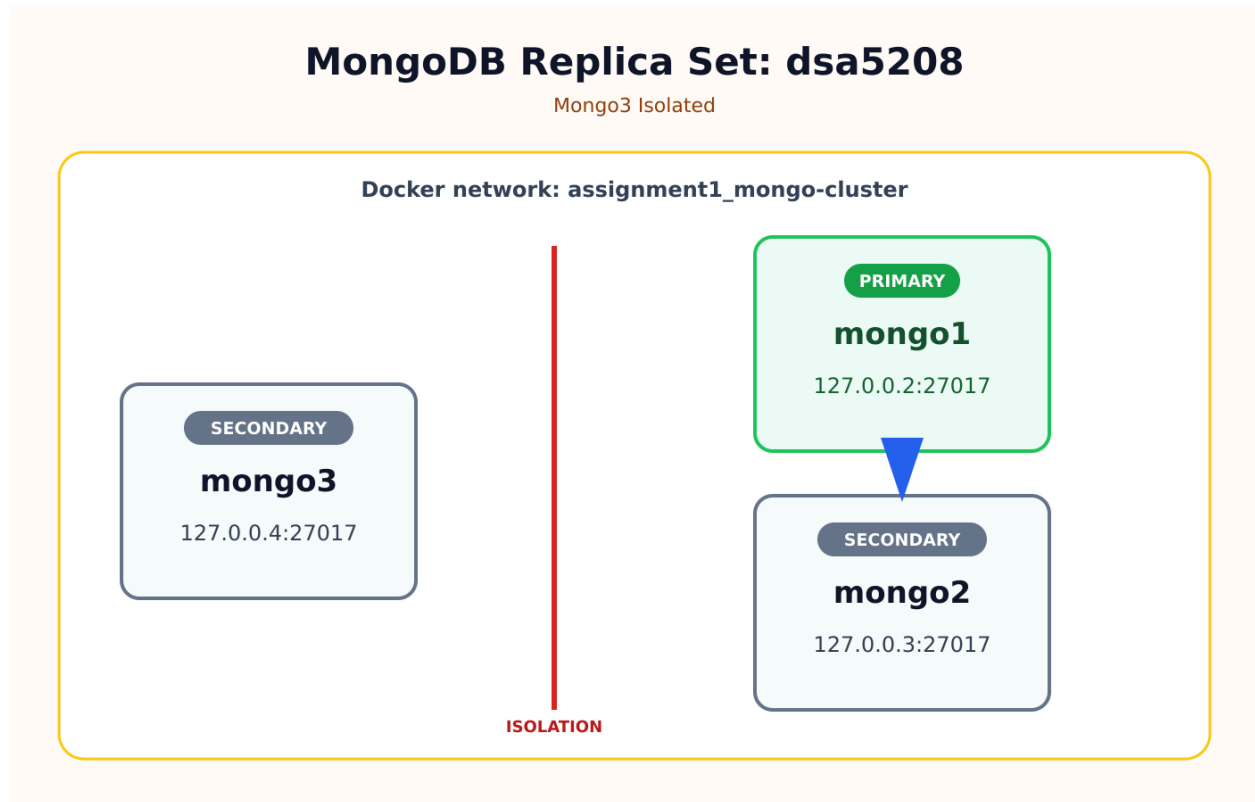


Figure 8: Replica Set with Network Partitioning on Secondary Node - Mongo3

`iptables` rules isolated **mongo3** from **mongo1** and **mongo2** while keeping all nodes running and accessible to the client. **mongo1** remained primary, and **mongo2** continued receiving updates. Both **majority/majority** and **w:1/local** were tested.

Before each test, the data was reset and the partition was established. Updates were sent through the primary, while secondary reads could reach either **mongo2** or the isolated **mongo3**. After each test, the firewall rules were removed and **mongo3** was checked for convergence.

7.2.1 Predictions

Table 28 presents the predictions. Each row applies to **both configurations** in this isolated-secondary scenario.

Consistency property	Without a causal session	With a causal session
----------------------	--------------------------	-----------------------

Read-your-writes	✗ May fail: mongo3 could return pre-write data	✓ Expected to hold; reads may time out
Monotonic reads	✗ May fail: reads could return older values after newer ones	✓ Expected to hold; reads may time out
Monotonic writes	✓ Expected to hold: both writes go sequentially to mongo1	Not tested
Writes-follow-reads	A stale course read could produce an outdated note; this alone would not establish a formal violation	After observing "CANCELLED", subsequent reads should return "CANCELLED" or time out

Table 28: Predictions with an Isolated Secondary

7.2.2 Read-Your-Writes Results

Initially, all nodes stored `student_status: "Undergraduate"`. While mongo3 remained isolated, the client tested two stages:

- **Without a causal session:** write `"Graduate"` through the primary, receive acknowledgement, then read from the secondaries.
- **With a causal session:** write `"Post Doc"` through the primary within a new causal session, receive acknowledgement, then read from the secondaries within that same session.

Different values distinguish the two writes. Each stage checks whether reads reflect **the update acknowledged in that stage**. Table 29 summarises the observed results.

Session setting	Configuration (write/read)	Expected value	Observed reads	Matches prediction?
No causal session	Majority/majority	<code>"Graduate"</code>	Both <code>"Undergraduate"</code> and <code>"Graduate"</code>	✓ Yes—violation observed

No causal session	w: 1/local	"Graduate"	Both "Undergraduate" and "Graduate"	✓ Yes—violation observed
Causal session	Majority/majority	"Post Doc"	"Post Doc" or timeout	✓ Yes—no violation observed
Causal session	w: 1/local	"Post Doc"	"Post Doc" or timeout	✓ Yes—no violation observed

Table 29: Read-your-writes results

7.2.1 Result Summary

Both configurations matched the predictions. Without a causal session, mongo3 could return "Undergraduate" despite the acknowledged "Graduate" update, violating read-your-writes.

With a causal session, completed reads returned "Post Doc". Reads unable to satisfy the session requirement timed out instead of returning older data. These timeouts were not consistency violations. After reconnection, mongo3 caught up to "Post Doc".

7.2.3 Monotonic-Reads Results

Initially, all nodes stored `seniority: "Year 1"`. While mongo3 remained isolated, the client updated the value sequentially to "Year 5" through the primary, then tested two read sequences:

- **Without a causal session:** read repeatedly from the secondaries, checking whether an older value appeared after a newer one.
- **With a causal session:** start a new causal session and repeat the secondary reads within that session, checking the same condition.

Each sequence was assessed separately. Once it returned "Year 5", a later "Year 1" would violate monotonic reads. Table 30 summarises the observed results.

Session setting	Configuration (write/read)	Observed reads	Backward reads?	Timeouts ?	Matches prediction?
No causal session	Majority/majority	"Year 1" returned after "Year 5"	Yes	No	✓ Yes—violation observed
No causal session	w: 1/local	"Year 1" returned after "Year 5"	Yes	No	✓ Yes—violation observed
Causal session	Majority/majority	"Year 5" or timeout	No	Yes	✓ Yes—no violation observed
Causal session	w: 1/local	"Year 5" or timeout	No	Yes	✓ Yes—no violation observed

Table 30: Monotonic-reads Results

7.2.3.1 Result Summary

Both configurations matched the predictions. Without a causal session, returning "Year 1" after "Year 5" violated monotonic reads because the client had already observed the newer value.

With a causal session, the first read returned "Year 5". Later reads returned "Year 5" or timed out, so no backward reads occurred. After reconnection, mongo3 caught up to "Year 5".

7.2.4 Monotonic-Writes Results

Initially, all nodes stored `academic_history: ["B.Sc. Computer Science"]`. No explicit causal session was used. The experiment followed three steps:

- **Write sequentially:** while mongo3 remained isolated, append "M.Sc. Data Science" and then "PhD Information Systems" through mongo1.

- **Read during the partition:** compare the histories returned by the primary and secondaries.
- **Check recovery:** remove the partition and sample mongo3's history to check whether the entries appeared in the correct order.

Table 31 abbreviates the academic qualifications as B.Sc., M.Sc., and PhD. The expected final order was [B.Sc., M.Sc., PhD].

Configuration (write/read)	Write outcomes	Secondary reads during partition	mongo3 during recovery	Matches prediction?
Majority/majority	Both acknowledged	Both [B.Sc.] and [B.Sc., M.Sc., PhD]	[B.Sc.] → [B.Sc., M.Sc., PhD]	✓ Yes—no violation observed
w: 1/local	Both acknowledged	Both [B.Sc.] and [B.Sc., M.Sc., PhD]	[B.Sc.] → [B.Sc., M.Sc., PhD]	✓ Yes—no violation observed

Table 31: Monotonic-Writes Results

7.2.4.1 Result Summary

Both configurations matched the predictions. The client issued the M.Sc. write before the PhD write through mongo1, and mongo3 contained both entries in that order after reconnection. Bachelor's-only reads during the partition indicated missing updates rather than reversed write ordering.

No incorrectly ordered history was observed. The intermediate [B.Sc., M.Sc.] state may have occurred between recovery samples.

7.2.5 Writes-Follow-Reads Results

Initially, all nodes stored `course_status: "ACTIVE"`. While mongo3 was isolated, the client changed the status to `"CANCELLED"` through the primary. mongo3 retained `"ACTIVE"`. The experiment then compared two stages:

- **Without a causal session:** record secondary course reads, then deliberately read "ACTIVE" from mongo3 and write an enrollment note based on that value.
- **With a causal session:** first read "CANCELLED" from the primary, then perform secondary course reads, write a note based on "CANCELLED", and read the note back—all within the same session.

Configuration (write/read)	Course reads without a session	Course reads with a causal session	Causal-session note read-backs	Matches prediction?
Majority/majority	Both "ACTIVE" and "CANCELLED"	"CANCELLED" or timeout	Session note containing "CANCELLED" or timeout	✓ Yes*
w: 1/local	Both "ACTIVE" and "CANCELLED"	"CANCELLED" or timeout	Session note containing "CANCELLED" or timeout	✓ Yes*

Table 32: Writes-Follow-Reads Results

Matches the predicted stale-read behaviour; no formal writes-follow-reads violation was demonstrated.

7.2.5.1 Result Summary

Both configurations matched the predicted stale-read behaviour. Without a causal session, the client wrote an outdated note based on "ACTIVE". This was not a formal writes-follow-reads violation because the primary accepting the note had already incorporated the "ACTIVE" state that the client read.

Within the causal session, completed course reads returned "CANCELLED" and completed note read-backs returned the session's note; other attempts timed out. This protection followed the session's initial observation of "CANCELLED"—it did not mean that a new session would automatically read the latest value. After reconnection, mongo3 contained "CANCELLED" and the session note.

7.3 Isolated Primary

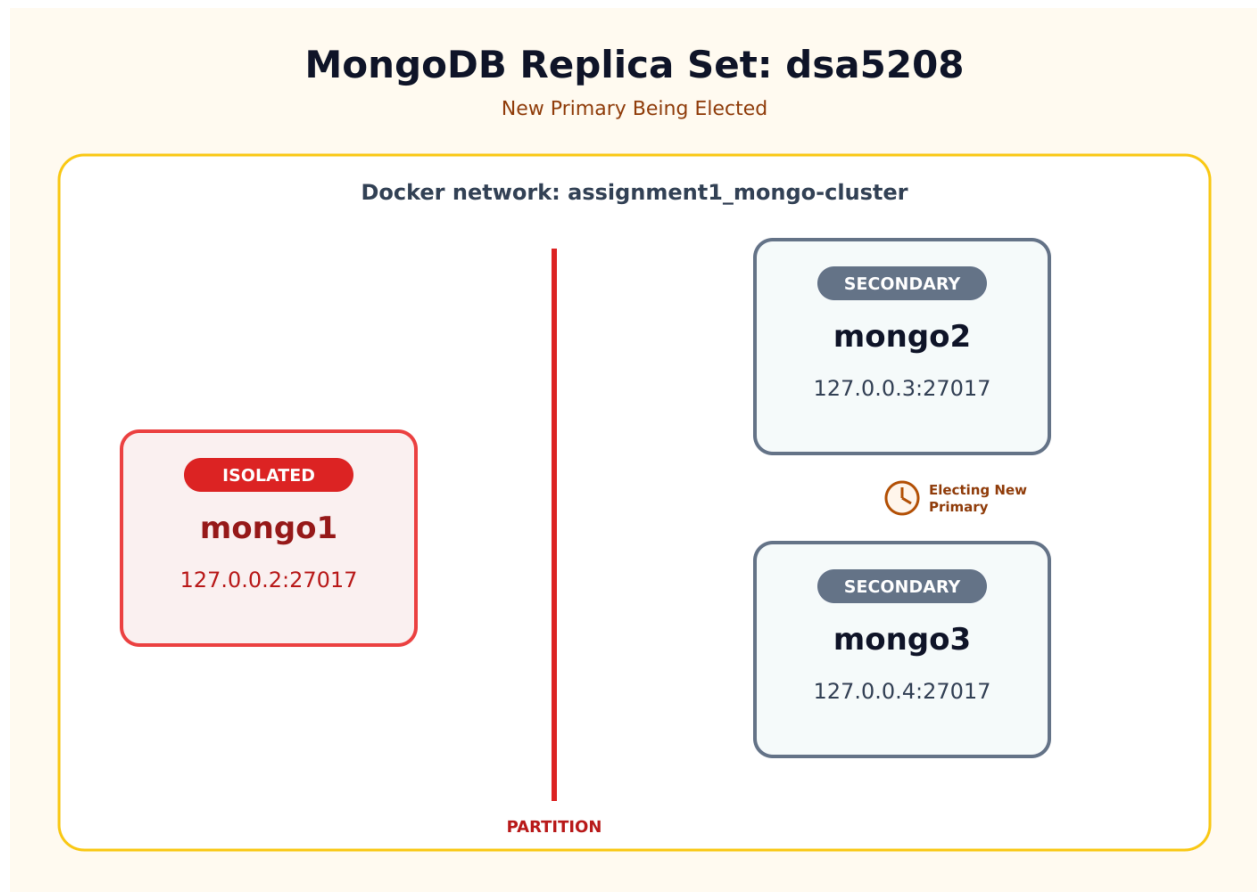


Figure 9: Replica Set Isolating the Primary Node

This experiment used `iptables` rules to isolate the current primary from both secondaries while keeping all nodes accessible to the client. The connected secondaries could elect a new primary, while the isolated primary would eventually step down.

Initially, the academic-history array contained "B.Sc. Computer Science". The experiment followed three stages:

- **Before the isolated primary stepped down:** attempt to append "M.Sc. MINORITY SIDE" directly to it.
- **After a new primary was elected:** append "M.Sc. MAJORITY SIDE" through the new primary.
- **After reconnection:** inspect the old primary's history and check its logs for rollback.

The two M.Sc. values distinguish writes made on opposite sides of the partition.

7.3.1 Predictions

Configuration (write/read)	Expected isolated-primary write outcome	Expected recovery
Majority/majority	Not acknowledged because the isolated primary cannot obtain majority confirmation	The minority-side entry is rolled back; the majority-side history is retained
w: 1/local	Acknowledged if the write reaches the isolated primary before it steps down	The acknowledged minority-side entry is rolled back; the majority-side history is retained

Table 33: Predictions with Isolated Primary

After stepping down, the isolated node was expected to reject further writes because it was no longer primary.

7.3.2 Results

Table 34 summarises the saved notebook outputs. Both configurations produced different histories on the two sides before reconnection.

Configuration (write/read)	Isolated - primary write	New - primary write	Old primary after reconnection	Rollback detected	Matches prediction
Majority/majority	Not confirmed before timeout	Not confirmed before timeout, but subsequently observed on both majority-side nodes	B.Sc. entry and "M.Sc. MAJORITY SIDE"; minority-side entry absent	Yes	✓ Yes
w: 1/local	Acknowledged	Acknowledged	B.Sc. entry and "M.Sc. MAJORITY SIDE"; minority-side entry absent	Yes	✓ Yes

Table 34: Results with Isolated Primary

Direct reads confirmed that the isolated primary stored "M.Sc. MINORITY SIDE" in both configurations, including when majority acknowledgement timed out. After the isolated primary stepped down, an additional write attempt returned `NotPrimaryError`.

Following reconnection, the old primary adopted the majority-side history. Its "M.Sc. MINORITY SIDE" entry disappeared, and "M.Sc. MAJORITY SIDE" remained.

7.3.3 Result Summary

Both configurations matched the predicted rollback behaviour. Under `w: 1/local`, the isolated primary acknowledged its write, but that entry disappeared after reconnection. Losing this acknowledged entry while retaining the later majority-side write demonstrated read-your-writes and monotonic-write violations under the report's evaluation criteria.

Under majority/majority, the isolated write was applied locally but never received majority acknowledgement. Its rollback therefore removed an unacknowledged update.

The results also showed that **a timeout did not determine the final write outcome**. In the majority/majority run, the isolated-primary write timed out and was rolled back, whereas the new-primary write also timed out but survived recovery.

7.4 Interpretation and Comparison with Predictions

The results generally matched the predictions. Isolating a secondary affected read freshness, while isolating the primary exposed the risk of losing acknowledged writes. Table 35 summarises the findings.

Scenario / property	Main observation	Matches prediction?	Additional note
Isolated secondary: read-your-writes	Both configurations returned stale values without a causal session. Causal reads returned the updated value or timed out.	✓ Yes	Majority acknowledgement alone did not ensure that secondary reads reflected the client's write.
Isolated secondary: monotonic reads	Both configurations produced backward reads without a causal session. No backward reads occurred within causal sessions.	✓ Yes	Some causal reads timed out while mongo3 remained isolated.
Isolated secondary: monotonic writes	Both configurations preserved the qualification order after reconnection.	✓ Yes	Both writes were issued sequentially to mongo1; no primary change occurred.
Isolated secondary: writes-follow-reads	Non-session notes used stale "ACTIVE" data. Causal sessions preserved the previously observed "CANCELLED" state.	✓ Yes, for stale-read behaviour	An outdated note did not establish a formal writes-follow-reads violation.
Isolated primary: majority/majority	The minority-side write was unacknowledged and later rolled back.	✓ Yes	The new-primary write also timed out but survived recovery.
Isolated primary: w: 1/local	The minority-side write was acknowledged and later rolled back.	✓ Yes	The client lost an acknowledged update while the later majority-side write survived.

Table 35: Comparison of Predictions and Observations with Network Partitions

The isolated-secondary results showed that a running but disconnected node could continue serving outdated data. Causal sessions prevented stale or backward reads relative to operations already observed within the session, although some reads could not complete during the partition. These observations applied to both configurations in this scenario.

The isolated-primary results exposed their different acknowledgement requirements. **w: 1** allowed the isolated primary to acknowledge a write before stepping down, but that write was later lost. Majority write concern could not be satisfied on the isolated node, so the rolled-back write had not received the requested acknowledgement.

Together, the experiments show that **preserving read consistency and protecting acknowledged writes are separate concerns**. They also confirm that a timeout leaves the write outcome uncertain: a timed-out write may later survive or be rolled back. Matching the predictions therefore does not mean that every consistency property was preserved.

8. Cross-Scenario Discussion

8.1 Comparison of the Four Configurations

All four configurations were tested under replication delay. The node-failure and network-partition experiments compared only **majority/majority** and **w: 1/local**. Conclusions about the other two configurations are therefore limited to replication delay. Table 36 summarises the main differences observed without causal sessions.

Configuration (write/read)	Read behaviour during replication delay	Write ordering during replication delay	Failure and partition findings
Majority/majority	Read-your-writes and monotonic-read violations occurred.	No violation observed.	Isolated writes could be applied locally but remained unacknowledged and were later rolled back.
Majority/local	Read-your-writes and monotonic-read violations occurred.	No violation observed.	Not tested.
w: 1/majority	Read-your-writes violations occurred. The monotonic-reads sequence remained at "Year 1", without regression.	No violation observed.	Not tested.
w: 1/local	Read-your-writes and monotonic-read violations occurred.	No violation observed.	Acknowledged writes could disappear after primary failure or an isolated-primary partition.

Table 36: Comparison of the Four Configurations Across the Experiments

These results show that majority read concern did not ensure that secondary reads reflected the client's latest write or previously observed value. Write ordering remained intact in the

replication-delay tests because the updates were issued sequentially to one primary and no failover occurred.

The difference between majority and **w: 1** acknowledgement became apparent when the primary's unreplicated writes were rolled back. Under **w: 1**, the client had already received acknowledgement. Under majority write concern, the corresponding write had not received the requested confirmation.

8.2 Effect of Causally Consistent Sessions

Causal sessions changed the observed read behaviour during replication delay and isolated-secondary partitions. Table 36 summarises their effects and the limits of the evidence.

Property	Observed effect	Limit of the evidence
Read-your-writes	Completed session reads reflected the session's update; some attempts timed out.	These tests did not involve primary failure or rollback.
Monotonic reads	No backward reads occurred within the recorded causal-session sequences.	A new session could initially read an older value before observing a newer one.
Monotonic writes	No explicit causal-session comparison was conducted.	The results cannot establish an additional benefit from sessions.
Writes-follow-reads	In the partition test, reads preserved the session's initial observation of "CANCELLED" .	The non-session stale note was not a formal writes-follow-reads violation. The replication-delay test also provided limited evidence because "ACTIVE" never changed.

Table 37: Effect of Causal Consistency Sessions Across the Experiments

Causal sessions linked later operations to earlier operations within the same session. When a secondary could not satisfy that requirement, some reads timed out instead of returning older data.

However, a causal session did not automatically obtain the latest database state. In the partition writes-follow-reads test, the session first read **"CANCELLED"** from the primary before testing secondary reads. The successful observations under **w: 1/local** therefore apply to these particular scenarios and do not establish protection against rollback.

8.3 Consistency, Availability, and Durability Trade-Offs

The experiments revealed three distinct concerns: whether reads respected the client's history, whether operations completed during disruption, and whether acknowledged writes survived recovery.

Experimental observation	Implication
Non-session secondary reads completed but could return stale or older values.	Completing a read did not ensure client-centric consistency.
Causal reads avoided the tested stale or backward observations, but some timed out.	Preserving session requirements could prevent reads from completing during disruption.
Majority acknowledgement timed out when the primary could not reach enough replicas.	Stronger acknowledgement requirements reduced write availability in those conditions.
w: 1 writes could be acknowledged before the isolated primary stepped down, then rolled back.	Successful acknowledgement did not ensure survival after failover.
Some timed-out writes survived recovery, while others were rolled back.	A timeout alone did not establish whether a write ultimately succeeded or failed.

Table 38: Trade-offs Observed Across Experiments

The disruption's location mattered. With an isolated secondary, the connected majority continued accepting writes, while the isolated node could serve stale reads. With an isolated primary, a write could temporarily exist only on the minority side and later disappear.

Overall, the experiments show that read consistency, operation completion, and write durability must be assessed separately. A configuration that behaved successfully during replication delay did not necessarily provide the same protection during primary failure or rollback.

9. Limitations

9.1 Single-Machine Docker Deployment

All three replica-set members ran in Docker containers on one machine. This provided a controlled environment but did not reproduce separate physical servers or real network conditions. The containers shared hardware resources and a host operating system, so the experiments could not assess independent machine failures or deployment across multiple locations.

9.2 Timing and Secondary-Selection Variability

Read outcomes depended on replication progress, operation timing, and which secondary served each request. Consequently, the frequency of stale reads and timeouts could vary between runs. The report therefore emphasises observed patterns rather than treating particular counts as repeatable outcomes.

Timing also affected writes attempted before an isolated primary stepped down. Recovery sampling could miss intermediate states, such as an academic history containing the bachelor's and master's entries before the doctoral entry appeared.

9.3 Scope of the Tested Failures and Partitions

The experiments covered controlled replication stalls, paused secondaries, primary termination, and partitions isolating one member. They did not examine every failure pattern, such as repeated network interruptions, disk failures, or simultaneous host failures.

All four concern configurations were tested under replication delay, but only majority/majority and `w: 1/local` were tested under node failures and network partitions. Explicit causal-session comparisons were limited to selected tests. The node-failure experiments also used primary reads, so their successful reads do not establish equivalent behaviour when reading from secondaries.

9.4 Observed Behaviour Versus a Guaranteed Property

A run with no observed violation does not prove that a configuration guarantees the property under all conditions. For example, consistently returning "Year 1" showed no monotonic-read violation, but did not test regression after observing a newer value. Similarly, successful causal-session reads under `w:1/local` during a secondary disruption do not establish protection against primary failure or rollback.

The writes-follow-reads experiments provided limited evidence. During replication delay, "ACTIVE" existed before the disruption and remained unchanged, so the test did not create a missing dependency. During the partition experiment, writing a note based on stale "ACTIVE" data demonstrated an application anomaly rather than a formal writes-follow-reads violation.

Finally, the replication-delay outputs showed a new enrollment note on mongo3 in several configurations despite the intended stall. The effectiveness of that disruption requires further verification before stronger conclusions can be drawn from those results.

10. Conclusion

This project evaluated four client-centric consistency properties in a three-node MongoDB replica set under replication delay, node failures, and network partitions. The results generally matched the scenario-specific predictions and showed that observed consistency depended on acknowledgement requirements, read location, session history, and the type of disruption.

Without causal sessions, secondary reads could violate read-your-writes and monotonic reads, including under majority/majority. Causal sessions prevented these violations in the recorded read sequences, although some reads timed out. Sequential write ordering was preserved during replication delay and isolated-secondary partitions, while the writes-follow-reads tests provided limited evidence of formal consistency protection.

Primary failure and isolated-primary partitions exposed the durability difference between the tested configurations. Acknowledged `w: 1` writes could be rolled back, whereas the corresponding majority writes did not receive the requested acknowledgement. Timed-out writes sometimes survived and sometimes disappeared, showing that a timeout alone did not determine the final outcome.

Overall, stronger write acknowledgement did not automatically ensure fresh secondary reads, and successful operation completion did not necessarily ensure durability. The findings support choosing consistency settings according to the application's read dependencies, acceptable waiting time, and tolerance for losing acknowledged updates, within the limitations of the tested scenarios.

References:

Docker. (2026). *Docker CLI reference*. Docker Docs. Retrieved September 10, 2026, from <https://docs.docker.com/reference/cli/docker/>

Factory. (n.d). *Fail points*. Factory AI. Retrieved September 10, 2026, from <https://factory.ai/open-source-wikis/mongo?page=systems%2Ffail-points.md>

GeeksforGeeks. (2026, April 30). *CAP theorem in system design*. Retrieved September 10, 2026, from <https://www.geeksforgeeks.org/system-design/cap-theorem-in-system-design/>

MongoDB. (n.d). *Find (database command)*. MongoDB Docs. Retrieved September 10, 2026, from <https://www.mongodb.com/docs/manual/reference/command/find/>

MongoDB. (n.d). *Hello (database command)*. MongoDB Docs. Retrieved September 10, 2026, from <https://www.mongodb.com/docs/manual/reference/command/hello/>

MongoDB. (n.d). *Insert (database command)*. MongoDB Docs. Retrieved September 10, 2026, from <https://www.mongodb.com/docs/manual/reference/command/insert/>

MongoDB. (n.d). *Update (database command)*. MongoDB Docs. Retrieved September 10, 2026, from <https://www.mongodb.com/docs/manual/reference/command/update/>

MongoDB. (n.d). *Deploying a MongoDB cluster with Docker*. Retrieved September 10, 2026, from <https://www.mongodb.com/resources/products/compatibilities/deploying-a-mongodb-cluster-with-docker>

MongoDB. (n.d). *Causal consistency and read and write concerns*. MongoDB Docs. Retrieved September 10, 2026, from <https://www.mongodb.com/docs/manual/core/causal-consistency-read-write-concerns/>

MongoDB. (n.d). *Delayed replica set members*. MongoDB Docs. Retrieved September 10, 2026, from <https://www.mongodb.com/docs/manual/core/replica-set-delayed-member/>

MongoDB. (n.d). *Read concern*. MongoDB Docs. Retrieved September 10, 2026, from <https://www.mongodb.com/docs/manual/reference/read-concern/>

MongoDB. (n.d). *Read preference*. MongoDB Docs. Retrieved September 10, 2026, from <https://www.mongodb.com/docs/manual/core/read-preference/>

MongoDB. (n.d). *Replica set configuration*. MongoDB Docs. Retrieved September 10, 2026, from <https://www.mongodb.com/docs/manual/reference/replica-configuration/>

MongoDB. (n.d). *Replica set elections*. MongoDB Docs. Retrieved September 10, 2026, from <https://www.mongodb.com/docs/manual/core/replica-set-elections/>

MongoDB. (n.d). *Write concern*. MongoDB Docs. Retrieved September 10, 2026, from <https://www.mongodb.com/docs/manual/reference/write-concern/>

PyMongo. (n.d). *Client sessions and causal consistency*. PyMongo documentation. Retrieved September 10, 2026, from <https://pymongo.readthedocs.io/>